

Kronos: Instantiation-Time Capability Gating for Multi-Tenant WebAssembly

John Abrahams and Ajax Li
University of Pennsylvania
Technical report, September 2026

Abstract—Kronos is a multi-tenant WebAssembly runtime built on Wasmtime. Before it instantiates a module, it checks the module’s declared imports against the capabilities of the tenant that owns the request. A module that imports a file or network function the tenant was not granted is refused, and none of its code runs. Kronos also bounds each invocation with fuel and a wall-clock deadline, and it runs guests compiled with POSIX threads through its own thread-spawn handler, which reports a worker trap as an error instead of exiting the host process. This report describes the design at a fixed commit, audits every path that instantiates a module, and reports measurements from two benchmark runs on one machine. The import check was enforced on the server’s request path. Two library entry points skipped or weakened it, and the thread-spawn handler’s abort mechanism could expire the deadlines of other tenants’ invocations. Memory grew by about 0.15 MiB per added isolate, against 0.21 to 0.23 MiB for a Wasmtime Go embedding and 2.2 MiB for wazero in comparable harnesses, while peak burst throughput was 7 to 12 times lower than either. A four-thread matrix multiply ran 4.8 to 6.6 times faster than a build that called the host for every element and 8.5 to 64 times slower than a plain single-threaded module, so the runs show no parallel speedup. This report supersedes an April 2026 draft whose headline claims did not hold up under this review.

I. INTRODUCTION

Running code you did not write on a shared host is an old problem with a new source: programs generated by language models and executed without review. The usual way to contain such code is to run it in a container or microVM and rely on a separately configured boundary, such as a seccomp profile, a network namespace, or an egress rule, to keep it from reaching what it should not [1], [2], [3], [4], [5]. Frontier models can already find and exploit weaknesses in container configurations [6].

WebAssembly offers a different starting point. A module reaches host functionality only through the imports it declares, and the embedder decides which imports exist [7], [8]. WASI builds its system interface on this property and has no ambient authority [9]. Kronos applies a per-tenant policy to the import list. Before instantiating a module, it compares the module’s imports with the capabilities the tenant holds and refuses the module if any import needs a capability the tenant lacks. We call this *deny by construction*: the decision is made once, against the module’s declared interface, before any guest code runs.

This report does three things:

- It describes the Kronos runtime at commit 2b73f3cb8 (July 2026): the import surface, the instantiation-time

check, metering, and the thread-spawn handler (Section III).

- It audits the code paths that instantiate modules and reports where the check held and where it did not (Section IV).
- It reports memory, throughput, and threaded-execution measurements from two committed benchmark runs, along with the limits of those runs (Section V).

An April 2026 draft of this work reported numbers and properties that this review could not reproduce from the committed data or confirm in the code. Section VII lists the corrections. The code described here has since been removed from the Kronos product repository, which now uses a different authority model. The commits cited in this report remain in its history.

II. BACKGROUND AND THREAT MODEL

A. WebAssembly imports and budgets

A WebAssembly module declares every function, memory, table, and global it needs from outside. Instantiation binds each declaration to a definition the embedder supplies [10]. Guest code cannot address memory outside its own linear memory, and its control flow is structured, so the import list describes everything a module can ask the host to do [7]. Wasmtime compiles modules with Cranelift [11], [12] and offers two execution budgets: fuel, which the guest consumes as it executes, and epoch interruption, which traps a guest at its next check once an externally advanced counter passes the deadline set on its store [13].

B. Threat model

The adversary is the guest module. Its code may be arbitrary, possibly written to attack the host, and it is admitted without human review. The host process holds state the guest must not reach: credentials for upstream services, other tenants’ stores, and any file or network destination the tenant was not granted. The trusted computing base is the Kronos host process, Wasmtime and the code it generates, and the host operating system. The intended properties are:

- P1 No unauthorized effects.** A module that imports a file or network function its tenant was not granted does not run.
- P2 Tenant independence.** A trap, panic, or timeout in one tenant’s invocation does not change the outcome of another tenant’s invocation.

TABLE I
HOST FUNCTIONS REGISTERED AT COMMIT 2b73f3cb8.

Group	Examples	Count	Gate
C++ runtime	<code>_Znwm, __cxa_throw</code>	11	none
Builtins	<code>memcpy, sin</code>	26	none
Token budget	<code>debit_tokens</code>	1	none
Intrinsics	<code>MAP, REDUCTION</code>	4	none
WASI-NN (stub)	<code>load, compute</code>	5	none
File and network	<code>__syscall_openat</code>	9	per name
WASI preview1	<code>fd_write</code>	46	filesystem

P3 Bounded work. Every invocation ends within its fuel and wall-clock budgets.

Microarchitectural side channels [14], [15] and exhaustion of shared host resources [16] are outside this model. Section VI returns to both.

III. DESIGN

A. Runtime structure

A `Kronos Runtime` owns one `Wasmtime Engine`, a background thread that advances the engine’s epoch every millisecond, and two linkers, one of which also defines the WASI preview1 functions. The server process shares a single `Runtime` across tenants, so compiled code is shared and isolation between tenants comes from the `Store`. Admission compiles an uploaded module once and registers trap stubs for any import the linkers do not define. Each invocation creates a fresh `Store` with the tenant’s fuel budget and deadline, instantiates the module, calls one export, and discards the store.

B. Import surface

Table I lists the host functions a tenant module could import: 56 outside WASI and 46 WASI preview1 functions, plus `thread-spawn` on the threaded path. Only two groups require a capability. The file and network group is classified by name: 22 names are gated, 9 of them have real handlers, and the rest resolve to trap stubs. Filesystem names require `AllowFilesystem`, networking names require `AllowNetworking`, and `close`, `read`, and `write` accept either. A module that imports any WASI preview1 function needs a WASI context, which the server creates only for tenants holding `AllowFilesystem`. Every other function, including the inference stubs and the intrinsics, is available to every tenant. The capability set contains only these two flags.

C. Instantiation-time check

Before it creates a store, the runtime calls `check_gated_imports`. The function walks the module’s import list, skips imports outside the `env` module, classifies each remaining name, and returns `CapabilityDenied` if the tenant lacks the capability a name requires. The linkers define every file and network handler for every tenant and stub unknown names, so the

TABLE II
INSTANTIATION PATHS AND WHETHER THE CAPABILITY CHECK RAN.

Entry point	Check	Capabilities used
Server /invoke	before store	the tenant’s
Library instantiate	before store	the runtime’s configura- tion
Library invoke, modules with only function imports	skipped	none checked
Library threaded instantiation	before store	filesystem forced on

linker never refuses a module. This pre-check is the only point of refusal.

Handlers enforce their own limits at call time. File paths are joined under the tenant’s `fs_root` after leading slashes are stripped, and any parent-directory component is refused. A `connect` succeeds only for an address and port on the tenant’s `allowlist`, which is empty by default.

D. Metering

Fuel and epoch interruption are enabled for every store. Fuel bounds the instructions a guest executes, and the epoch deadline bounds the wall-clock time it spends in guest code. Neither interrupts a guest while it is blocked inside a host function, so each host function must bound its own blocking. As of July 2026 the server stores the fuel each invocation consumed in its SQLite database. Nothing uses that record for billing.

E. Threaded guests

Kronos runs guests compiled with `clang -pthread` against `wasi-libc` [17]. `Wasmtime` stores cannot be shared across threads, so each worker gets its own store, and all workers bind the same shared memory. The `wasmtime-wasi-threads` crate available at the time exited the host process when a spawned thread trapped [11], which is unacceptable when many tenants share the process. Kronos registers its own `thread-spawn` function instead. It runs each worker inside `catch_unwind` on Rayon’s global pool [18], reuses worker stores from a per-isolate pool, and on a trap sets a per-isolate abort flag and records the first error. The guest header used in the benchmarks fixes the worker count at four and falls back to sequential loops below a size threshold (256 elements for map steps, 4096 for reductions).

To stop workers that are still running after a trap, the handler advances the engine’s epoch by the deadline length plus one tick. Section IV-C explains why that breaks tenant independence.

IV. AUDIT

We read every path through which a module could be instantiated at commit 2b73f3cb8. Table II summarizes the result.

A. The server path

For `/invoke`, the server creates a WASI context only when the tenant holds `AllowFilesystem` and then calls the checked instantiation path, which runs `check_gated_imports` before creating the store. On this path the code enforces P1 for the file and network names and for WASI preview1. The server does not call `Runtime::invoke` or the threaded path.

B. Paths that skip or weaken the check

`Runtime::invoke` prepares a pre-resolved instance template (`InstancePre`) for any module whose imports are all functions, and it instantiates such modules through that template without calling the check. A module that defines its own memory and imports `__syscall_socket` and `__syscall_connect` qualifies. Its code runs, and its `connect` calls are refused only at call time, unless the runtime was configured with an allowlist. P1 does not hold on this path. The server never used it, but the library exposed it as the general-purpose call.

The threaded path sets `AllowFilesystem` to true before running the check, because it always links WASI. A tenant without the filesystem capability could therefore instantiate a threaded module that imports file functions. Networking stayed gated on this path.

C. Abort interference

Every store on a `Runtime` belongs to the same engine and therefore reads the same epoch counter, and each store’s deadline is set relative to that counter. When a worker traps, the handler advances the shared counter by the deadline length plus one tick. Every other in-flight store whose deadline length is no longer than the aborting tenant’s is pushed past its deadline and traps at its next epoch check. One tenant’s faulty threaded module can therefore fail every concurrent invocation on the same runtime, which violates P2. The threaded path uses the runtime’s shared engine, not a private one. The server did not expose this path, so the effect was limited to library use and benchmarks.

D. Other observations

- The `socket` handler allocates a descriptor without checking any capability. For modules that skip the check, the `connect` allowlist is the only barrier.
- Path resolution does not resolve symbolic links, so a link placed inside a tenant’s `fs_root` by the host would let file calls leave it.
- The WASI-NN functions are registered for every tenant and report that inference is unsupported.

The fixes follow from the findings: send every entry point through the check, derive the threaded path’s capabilities from the tenant, and stop aborted workers with a per-isolate signal instead of the shared epoch counter. The code at `2b73f3cb8` has none of these fixes.

TABLE III

RESIDENT MEMORY ADDED PER ISOLATE, $N = 100$ TO 200 (MiB).

Runtime	Run A	Run B
Kronos	0.15	0.15
Wasmtime (Go embedding)	0.23	0.21
wazero	2.19	2.22

TABLE IV

PEAK BURST THROUGHPUT, REQUESTS PER SECOND (AT N).

Runtime	Run A	Run B
Kronos	14,365 (64)	14,435 (64)
Wasmtime (Go embedding)	103,237 (512)	164,388 (512)
wazero	170,637 (256)	157,207 (256)

V. EVALUATION

A. Setup and provenance

All measurements come from two benchmark runs committed on April 28, 2026, about an hour apart: run A (commit `2b3bdd40a`) and run B (commit `9d58f5da5`). The outputs do not name the machine. Their timer readings fall on multiples of 41.67 ns, the tick of the monotonic clock on Apple Silicon Macs, so both runs were taken on an Apple Silicon Mac. The core count was not recorded. The Kronos harness is a Rust program. The Wasmtime and wazero harnesses are Go tests that use the Wasmtime Go embedding and wazero [19]. Trials per point range from two to eight, so small differences are not meaningful. The same commits contain a Docker baseline that ran on a different machine and used a different memory metric, and we do not report it. The runtime measured here is the April 2026 code. The gating code examined in Section IV changed substantially between April and July.

B. Memory per isolate

Each harness creates N isolates, where an isolate is a fresh runtime, compiled module, and instance of a small recursive-Fibonacci module, and it reports the process’s resident set size above a baseline. Table III gives the growth per added isolate between $N = 100$ and $N = 200$, which excludes one-time setup costs.

The Wasmtime Go embedding used 1.4 to 1.5 times as much memory per added isolate as Kronos, and wazero used 14 to 15 times as much. Three caveats apply. Resident set size counts only pages a process has touched. The harnesses differ in language and allocator. A module this small understates the per-isolate cost of real workloads, whose linear memory would dominate.

C. Multi-tenant throughput

In this benchmark N threads each instantiate a private isolate and call `sfib(20)` once. Table IV reports the highest request rate across $N \in \{1, 4, 16, \dots, 1024\}$, averaged over three trials.

Kronos’s peak was 7 to 11 times lower than the Wasmtime Go embedding’s and 11 to 12 times lower than wazero’s.

TABLE V
MATRIX MULTIPLY UNDER KRONOS, RUN B (MEAN MS).

N	Intrinsic	Threaded (4)	Plain
32	7.11	1.49	0.023
64	34.7	5.56	0.151
96	116.8	18.5	0.503
128	277.6	42.0	4.91

Kronos repeated within 1% between runs, while the Wasmtime peak moved by 59%, so one run of this benchmark does not characterize a runtime. With a single tenant the runtimes were closer: 7,529 and 7,837 requests per second for Kronos, 8,811 and 6,540 for Wasmtime, and 23,392 and 21,486 for wazero. We did not profile Kronos’s invocation path, so this report does not attribute the gap to any particular cost.

D. Threaded execution

Run B measured three builds of an $N \times N$ double-precision matrix multiply under Kronos. The *intrinsic* build is the benchmark’s C++ kernel compiled without `-pthread`. Its map and reduce steps call host intrinsics, which costs about three guest-host crossings per inner element. The *threaded* build compiles the same C++ source with `-pthread` and different flags, runs four guest threads, and keeps its inner loops inside the guest. The *plain* build is a hand-written single-threaded WebAssembly text module that computes the same product with no host calls.

The threaded build was 4.8 to 6.6 times faster than the intrinsic build (Table V), and the April draft reported that ratio as a parallel speedup. It was also 8.5 to 64 times slower than the plain module. Most of the difference between the intrinsic and threaded builds comes from removing host calls, and nothing in these runs isolates a gain from the extra threads. The k -means and SVD kernels use the same wrappers and share the confound, and no plain baseline was recorded for them. What the runs do show is that threaded guests ran to completion through the custom handler. One result is unexplained: the plain module’s time grew 9.8 times from $N = 96$ to $N = 128$, where the arithmetic grows 2.4 times, while the Wasmtime Go embedding’s time on the same module grew 2.5 times (0.36 to 0.89 ms).

VI. RELATED WORK

a) WebAssembly isolation: RLBox uses WebAssembly to sandbox libraries inside Firefox, with a type-checked interface at the boundary [20]. Bosamiya et al. compile WebAssembly to native code with provable sandboxing [21]. WaVe verifies that a WASI runtime’s host calls preserve memory safety and restrict file and network access [22], which makes it the closest verified counterpart to the file and network handlers here. Swivel hardens WebAssembly against Spectre [14]. Kronos inherits its memory isolation and side-channel posture from Wasmtime [8].

b) Capabilities: Capability-based protection goes back to Dennis and Van Horn [23]. Capsicum brought capability mode to UNIX [24], and Miller’s object-capability work

treats authority as something passed explicitly [25]. WASI already has no ambient authority [9]. Kronos adds a narrow layer on top: a per-tenant policy over non-WASI host imports, checked before instantiation. Earlier drafts called Kronos a “Type II exovisor” after exokernels [26]. We dropped the term because Kronos exposes operating-system abstractions such as file descriptors and libc functions, while an exokernel exposes hardware resources.

c) Density and serverless WebAssembly: Lucet reported instantiating modules in under 50 microseconds with a few kilobytes of overhead and running tens of thousands of them in one process [27]. Sledge is a WebAssembly runtime for serverless functions at the edge [28]. Faasm isolates functions as Faaslets using WebAssembly memory safety together with cgroups and network namespaces, at about 200 KB per Faaslet [29]. GRANNY, its successor, runs OpenMP and MPI applications as WebAssembly and is the relevant comparison for any threaded path [30]. Yu et al. show that WebAssembly guests can exhaust host resources shared across tenants [16], an attack that fuel and epoch budgets do not address.

d) Sandboxes for generated code: Services that run generated code, such as E2B, Modal, and Daytona [3], [4], [5], build on microVMs and hardened containers such as Firecracker and gVisor [1], [2]. Andronchik and Lokhmakov compare the engines behind five of them [31]. These systems run arbitrary Linux programs. Kronos runs only WebAssembly modules, which limits what it can execute and is what makes an import check possible. Cloudflare Workers run many tenants as V8 isolates in one process [32], and MVVM runs agent workspaces in WebAssembly alongside hardware enclaves [33].

VII. CORRECTIONS TO THE APRIL 2026 DRAFT

An April 2026 draft titled “Khronos: A Type II Exovisor Architecture with WebAssembly” was posted publicly. This review corrects the following claims.

- **Memory.** The draft reported 0.08 MiB per isolate at $N = 200$. That value is the $N = 1$ row; growth per added isolate was about 0.15 MiB. Its ratios of 3 times denser than Wasmtime and 18 to 20 times denser than Docker become 1.4 to 1.5 times against the Wasmtime Go embedding, and the Docker figure is not comparable.
- **Throughput.** The draft’s 38,000 requests per second for Kronos does not appear in the run behind its other numbers, where Kronos peaked at 14,365. Its 41,000 for an optimized configuration has no committed source.
- **Parallel speedup.** The 6.7-times speedup on four cores compared the threaded build with the host-call-heavy intrinsic build (Section V-D).
- **Import surface.** The draft counted about 74 host entry points. The code registers 56 outside WASI and 102 including WASI preview1.
- **Gating.** The draft said the inference and intrinsic imports were gated and that the linker rejected ungranted imports. Neither was true (Section III-C).
- **Tenant independence.** The draft said parallel invocations used a private engine, so an abort could not affect other

tenants. They used the shared engine (Section IV-C).

- **Hardware.** The draft attributed its measurements to a 4-vCPU cloud VM. The outputs indicate an Apple Silicon Mac.

VIII. LIMITATIONS AND STATUS

- **The audit is a code reading.** We did not build exploit modules for the paths in Section IV, and we did not rerun the repository’s network-refusal demonstration for this report.
- **The measurements are thin.** There are two runs on one machine, two to eight trials per point, a single small module for memory and throughput, and harnesses written in different languages.
- **Guests must be WebAssembly.** Kronos cannot run an arbitrary Linux program, such as a Python interpreter with native extensions.
- **The code is historical.** At commit d02d71a88 the Kronos product repository no longer contains the intrinsics, the custom thread-spawn handler, or the check described here. It pins its guest interface at 40 `env` functions and 46 WASI preview1 functions and grants file access through WASI preopened directories.

IX. CONCLUSION

Checking a module’s imports against a tenant’s capabilities before instantiation takes little code in a Wasmtime embedding, and on Kronos’s server path the code enforced it. The audit shows how easily the property is lost. A performance fast path skipped the check, a threading path weakened it, and an abort mechanism built on a counter shared by all tenants broke tenant independence. A runtime that depends on this kind of check needs one instantiation entry point that every caller uses and a test that asserts refusal on each path. On performance, Kronos used less memory per added isolate than the two Go-hosted runtimes in comparable harnesses, delivered much lower burst throughput, and showed no measured speedup from threads.

AVAILABILITY

The code and raw benchmark outputs are in the history of https://github.com/jca225/webAssembly_Interpreter. Raw outputs are under `benchmark/results/` at the run commits.

Runtime, audit	2b73f3cb8c7474a10939fbf69fc97b80073a029d
Run A	2b3bdd40a83a422c2ac23b4boda4a471d1b279d
Run B	9d58f5da5a4082692d4d6b32190c78f20dacf1a3

REFERENCES

- [1] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Pionka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020, pp. 419–434.
- [2] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “The true cost of containing: A gVisor case study,” in *Proc. 11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, Renton, WA, USA, 2019.
- [3] E2B, “E2B.” [Online]. Available: <https://e2b.dev/>. Accessed: Sep. 12, 2026.
- [4] Modal Labs, “Sandboxes,” Modal documentation. [Online]. Available: <https://modal.com/docs/guide/sandboxes>. Accessed: Sep. 12, 2026.
- [5] Daytona, “Secure infrastructure for running AI-generated code.” [Online]. Available: <https://www.daytona.io/>. Accessed: Sep. 12, 2026.
- [6] R. Marchand, A. O. Cathain, J. Wynne, P. M. Giavridis, S. Jennings, F. Tuxworth, T. H. Dur, S. Deverett, J. Wilkinson, J. Gwartz, and H. Coppock, “Quantifying frontier LLM capabilities for container sandbox escape,” arXiv:2603.02277, 2026.
- [7] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. F. Bastien, “Bringing the web up to speed with WebAssembly,” in *Proc. 38th ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200, doi: 10.1145/3062341.3062363.
- [8] Bytecode Alliance, “Security,” Wasmtime documentation. [Online]. Available: <https://docs.wasmtime.dev/security.html>. Accessed: Sep. 12, 2026.
- [9] WebAssembly Community Group, “WASI design principles,” WebAssembly/WASI repository, docs/DesignPrinciples.md. [Online]. Available: <https://github.com/WebAssembly/WASI/blob/main/docs/DesignPrinciples.md>. Accessed: Sep. 12, 2026.
- [10] A. Rossberg, Ed., “WebAssembly core specification (release 2.0),” W3C Candidate Recommendation Snapshot, Dec. 17, 2024. [Online]. Available: <https://www.w3.org/TR/2024/CR-wasm-core-2-20241217/>
- [11] Bytecode Alliance, “Wasmtime.” [Online]. Available: <https://wasmtime.dev/>. Accessed: Sep. 12, 2026.
- [12] Bytecode Alliance, “Craneflight.” [Online]. Available: <https://craneflight.dev/>. Accessed: Sep. 12, 2026.
- [13] Bytecode Alliance, “Config in wasmtime (consume_fuel, epoch_interruption),” wasmtime crate API documentation. [Online]. Available: <https://docs.rs/wasmtime/latest/wasmtime/struct.Config.html>. Accessed: Sep. 12, 2026.
- [14] S. Narayan, C. Disselkoen, D. Moghimi, S. Cauligi, E. Johnson, Z. Gang, A. Vahldiek-Oberwagner, R. Sahita, H. Shacham, D. Tullsen, and D. Stefan, “Swivel: Hardening WebAssembly against Spectre,” in *Proc. 30th USENIX Security Symp.*, 2021, pp. 1433–1450.
- [15] Z. Weissman, T. Tiemann, T. Eisenbarth, and B. Sunar, “Microarchitectural security of Firecracker VMM for serverless cloud platforms,” in *Information Systems Security (ICISS 2024)*, ser. Lecture Notes in Computer Science, vol. 15416. Cham, Switzerland: Springer, 2025, pp. 3–24, doi: 10.1007/978-3-031-80020-7_1.
- [16] Z. Yu, D. Zhan, L. Ye, H. Yu, H. Zhang, and Z. Tian, “Exploring and exploiting the resource isolation attack surface of WebAssembly containers,” in *Proc. 34th USENIX Security Symp.*, 2025, pp. 1111–1128.
- [17] WebAssembly Community Group, “wasi-threads.” [Online]. Available: <https://github.com/WebAssembly/wasi-threads>. Accessed: Sep. 12, 2026.
- [18] “Rayon: A data parallelism library for Rust.” [Online]. Available: <https://github.com/rayon-rs/rayon>. Accessed: Sep. 12, 2026.
- [19] “wazero.” [Online]. Available: <https://wazero.io/>. Accessed: Sep. 12, 2026.
- [20] S. Narayan, C. Disselkoen, T. Garfinkel, N. Froyd, E. Rahm, S. Lerner, H. Shacham, and D. Stefan, “Retrofitting fine grain isolation in the Firefox renderer,” in *Proc. USENIX Security Symp.*, 2020, pp. 699–716.
- [21] J. Bosamiya, W. S. Lim, and B. Parno, “Provably-safe multilingual software sandboxing using WebAssembly,” in *Proc. USENIX Security Symp.*, 2022, pp. 1975–1992.
- [22] E. Johnson, E. Laufer, Z. Zhao, D. Gohman, S. Narayan, S. Savage, D. Stefan, and F. Brown, “WaVe: A verifiably secure WebAssembly sandboxing runtime,” in *Proc. IEEE Symp. Security and Privacy (SP)*, 2023, pp. 2940–2955, doi: 10.1109/SP46215.2023.10179357.
- [23] J. B. Dennis and E. C. Van Horn, “Programming semantics for multiprogrammed computations,” *Commun. ACM*, vol. 9, no. 3, pp. 143–155, Mar. 1966, doi: 10.1145/365230.365252.
- [24] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, “Capicum: Practical capabilities for UNIX,” in *Proc. 19th USENIX Security Symp.*, 2010.
- [25] M. S. Miller, “Robust composition: Towards a unified approach to access control and concurrency control,” Ph.D. dissertation, Johns Hopkins Univ., Baltimore, MD, USA, 2006.
- [26] D. R. Engler, M. F. Kaashoek, and J. O’Toole, Jr., “Exokernel: An operating system architecture for application-level resource man-

- agement,” in *Proc. 15th ACM Symp. Operating Systems Principles (SOSP)*, 1995, pp. 251–266, doi: 10.1145/224056.224076.
- [27] P. Hickey, “Lucet takes WebAssembly beyond the browser,” Fastly Blog, Mar. 28, 2019. [Online]. Available: <https://www.fastly.com/blog/announcing-lucet-fastly-native-webassembly-compiler-runtime>
 - [28] P. K. Gadepalli, S. McBride, G. Peach, L. Cherkasova, and G. Parmer, “Sledge: A serverless-first, light-weight Wasm runtime for the edge,” in *Proc. 21st Int. Middleware Conf.*, 2020, pp. 265–279, doi: 10.1145/3423211.3425680.
 - [29] S. Shillaker and P. Pietzuch, “Faasm: Lightweight isolation for efficient stateful serverless computing,” in *Proc. USENIX Annual Technical Conf. (ATC)*, 2020, pp. 419–433.
 - [30] C. Segarra, S. Shillaker, G. Li, E. Mappoura, R. Bruno, L. Vilanova, and P. Pietzuch, “GRANNY: Granular management of compute-intensive applications in the cloud,” in *Proc. 22nd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2025, pp. 205–218.
 - [31] G. Andronchik and P. Lokhnikov, “AI code sandboxes: A comparative security study. Part 1 of 2: Engine-level properties,” arXiv:2606.08433, Jun. 2026.
 - [32] K. Varda, “Mitigating Spectre and other security threats: The Cloudflare Workers security model,” Cloudflare Blog, Jul. 29, 2020. [Online]. Available: <https://blog.cloudflare.com/mitigating-spectre-and-other-security-threats-the-cloudflare-workers-security-model/>
 - [33] Y. Yang, A. Hu, Y. Zheng, B. Zhao, X. Zhang, D. Xiang, K. Chu, W. Zhang, and A. Quinn, “MVVM: Deploy your AI agents – securely, efficiently, everywhere,” arXiv:2410.15894, 2024.